

Conversing with a Production Process

Interaction Guide

1 Introduction and Motivation

The integration of digital twins into modern manufacturing environments has revolutionized how factories monitor, simulate, and optimize production processes. However, interacting effectively with these systems often requires *advanced technical knowledge*. This poses significant challenges for factory operators who may not have expertise in formal methods or programming languages.

Tasks such as performing process mining analysis, running discrete event simulations of the production line, or verifying formal properties of the system rely on specific digital models and require familiarity with logical formalism like *Computation Tree Logic* (CTL)¹. This technical barrier not only limits accessibility but also hinders the effective utilization of production process intelligence in everyday factory operations.

To address this issue, we present a *conversational framework* for enabling workers to perform simulations, verify properties, execute process mining analyses, and pose multi-perspective queries in natural language, without requiring deep technical knowledge.

2 Case Study: The LEGO Factory

We consider the *LEGO Factory* as our reference environment, whose topology is reported in Figure 1. This is a simplified physical reproduction of a production line constructed using LEGO bricks, encompassing multiple stations, each responsible for processing tokens². Each station has a fixed **unary** capacity (i.e., only one piece at the time can be processed).

Data Availability. For this case study, the system features a three-hour execution event log, which records events from past executions of the production process. It is used to extract the parameters needed for simulation, verification (the semantics of the automaton events in Table 1), and process mining tasks over the process.

3 Supported Tasks and Capabilities

3.1 Simulation and Predictive Maintenance

The digital twin supports a simulation task to compute the number of pieces produced within a specific timeframe and can provide related statistics (e.g., processing, waiting, and transfer times for each station, as well as aggregate average statistics for the entire line).

To ensure valid results, it requires the user to specify three **mandatory** parameters from the user:

- **Simulation time:** The temporal horizon (duration) for which the simulation should run.

¹CTL is a logic where the model of time is a tree-like structure. The future is not determined; there are different possible paths. Model checkers use it to verify if a SW or HW system possesses safety or liveness properties.

²We define a “token” as an individual unit of product or material moving through the production process.

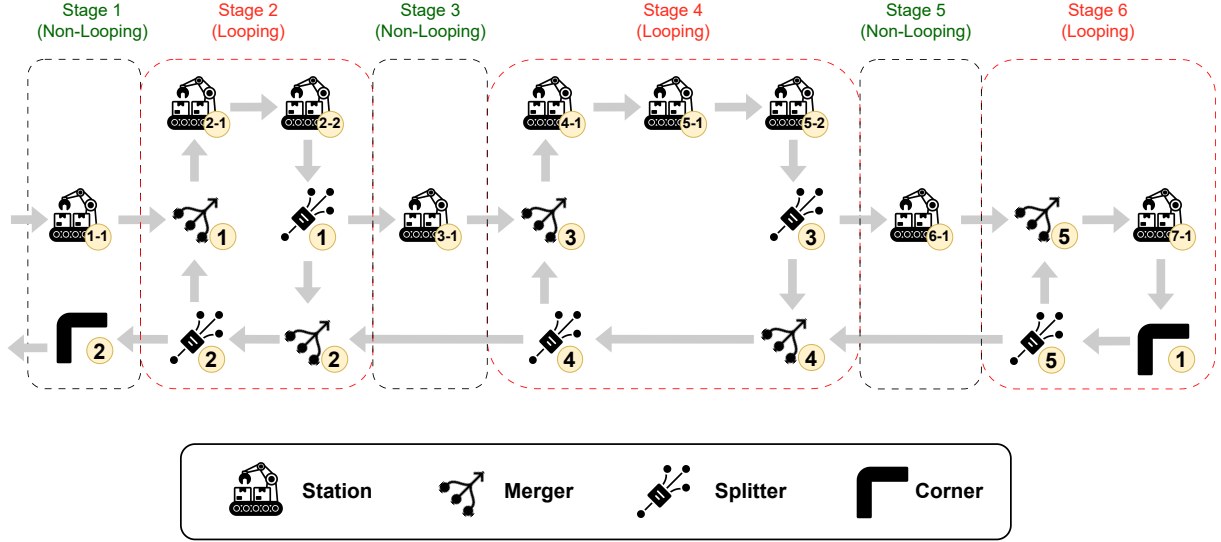


Figure 1: Schematic representation of the LEGO factory's topology

Symbol	Observable Event
s_1	corner1_START
s_3	corner1_TRANSFER
s_8	splitter1_FORWARD
s_{10}	splitter1_TRANSFER
s_{27}	station11_LOAD
s_{28}	station11_PROCESS
s_{30}	station11_UNLOAD
s_{65}	station51_LOAD
s_{68}	station51_PROCESS
s_{71}	station51_UNLOAD
s_{83}	station61_PROCESS
s_7	End

Table 1: Representative subset of observable events in the automaton

- **Number of replicas:** The number of simulation runs to execute in order to compute reliable average metrics.
- **Initial state:** Specifies whether to start from an *empty initial state* for the system (no work-in-progress) or after a *warm-up period* (e.g., running the line for a certain amount of *seconds* to create realistic queues before starting the actual simulation recording).

Optionally, the user can also specify a target **number of pieces** they would like to produce within that timespan (which triggers a proactive *feasibility* check) and/or simulate the **failure of a specific station**.

3.2 Formal Verification

The system enables verification via Uppaal³ of the factory's automaton using CTL.

Allowed tasks:

- **Semantic Lookup:** Retrieve the meaning/mapping of automaton locations and events.
- **Reachability:** Can a certain location be reached?

³Uppaal is a tool for modeling, validation, and verification of real-time systems modeled as networks of timed automata. Link: <https://uppaal.org/>

- **Safety/Deadlock:** Is the system deadlock-free?
- **Liveness:** If P happens, will Q eventually happen?
- **Time-Bounded:** Verification of properties within a timeframe (in seconds).

3.3 Process Mining

The system can analyze the historical event log to provide descriptive insights.

Allowed tasks:

- **Process Discovery:** Show the process Petri net discovered from the log.
- **Conformance Checking:** Check deviations between the log and the reference model.
- **Statistical Analysis:** List most frequent traces, start/end activities, or station usage.

4 Example Queries

Below are examples of supported questions according to the user profile and expertise.

Production Manager (Simulation & Predictive Maintenance)

As a **production manager**, I need to establish a performance baseline, plan a specific target production, and assess the impact of a machine breakdown.

1. *Run 5 simulation replicas starting from an empty initial state for a duration of 1000 seconds to check how many pieces are produced.*
2. *Perform 5 simulation replicas with a warm-up of 200 seconds. After the warm-up, simulate the production of 100 pieces and report if they can be produced in 500 seconds.*
3. *Simulate the production over 3000 seconds starting from an empty initial state with 5 replicas. If **station_51** fails, what is the mean processing time for producing 200 pieces?*

Safety Engineer (Formal Verification)

As a **safety engineer**, I want to audit the production system by understanding the automaton semantics (locations/events), verifying global safety (i.e., deadlocks), and checking specific temporal behaviors.

1. *Return the events and locations in the automaton.*
2. *What does location q_6 mean?*
3. *What does event s_6 mean?*
4. *Is event s_6 reachable?*
5. *Check if location q_6 is reachable within 15 seconds.*
6. *Check if the process contains any deadlocks.*
7. *Check if the process will always eventually reach location q_1 .*

Process Analyst (Process Mining)

As a **process analyst**, I have a raw event log and I want to discover the underlying Petri net, check if reality matches the model, and then analyze execution frequencies.

1. *Can you discover the Petri net from the provided event log?*
2. *Is the event log conformant with the behavior prescribed by the Petri net?*
3. *What is the most frequent execution variant recorded in the log?*
4. *What is the workload of `station_31` in the log?*

Plant Manager (Multi-Perspective Questions)

As a **plant manager**, I need to make a real-time decision by combining a safety check with a performance forecast in a single step.

1. *Please check whether there is a deadlock. Then, assuming `station_51` fails, run 10 simulation replicas for 15000 seconds from an empty initial state to check how many pieces can be produced.*

IMPORTANT:

- The current system **does not retain memory of previous interactions**. Therefore, do not refer to previous results (e.g., “based on the last answer...”), as the safety guardrails will block these requests to prevent non-factual outputs.
- Please ensure **each query is self-contained**, and assume the numerical/logical results are mathematically correct.
- If you encounter an **Error 503**, please wait a few moments before retrying. This error is caused by an overload of the Google API language model underlying the system.

The conversational system is available for testing at: <http://195.231.61.196:7860/>